웹 개발자 부트캠프 과정

CodingOn

Dependency Injection

잘 기억하고 있는지 복습해보죠! 😊

복습. DI (Dependency Injection, 의존성 주입)

- 구성 요소의 의존 관계가 소스코드 내부가 아닌 외부의 설정 파일을 통해 정의됨
- 즉, 외부에서 객체를 주입 받아 사용함
- DI (의존성 주입) 방법으로 IoC (제어의 역전) 를 구현

"이를 통해 코드 간의 재사용률을 높이고, 모듈 간의 결합도를 낮출 수 있음"

의존성이란?

- 게시글 save과 delete 기능 구현을 위해서는 FileBoardPersistence 가 필요!
- 즉, BoardService 클래스가 FileBoardPersistence 클래스에 의존함

```java
// BoardService
// - 게시판 목록을 관리하는 기능 제공
public class BoardService {
  private final FileBoardPersistence persistence;

  public BoardService() {
    this.persistence = new FileBoardPersistence();
  }

  public void save() {
    persistence.save();
  }

  public void delete() {
    persistence.delete();
  }
}
```

```java
// FileBoardPersistence
// - 파일에 게시판 목록을 저장하는 클래스
public class FileBoardPersistence {
  public void save() {
  }

  public void delete() {
  }
}
```

앞선 코드의 문제점은?

- 기획이 변경되어 파일이 아닌 데이터베이스에 게시글을 저장해야 한다!
- BoardService 클래스도 코드를 수정해야 함 ➡ FileBoardPersistence 클래스에 의존하는 클래스가 많을 수록 수정이 필요한 코드가 많아짐 ➡ 의존성 주입 등장 배경

```java
// BoardService
// - 게시판 목록을 관리하는 기능 제공
public class BoardService {
  private final DbBoardPersistence persistence;

  public BoardService() {
    this.persistence = new DbBoardPersistence();
  }

  public void save() {
    persistence.save();
  }

  public void delete() {
    persistence.delete();
  }
}
```

```java
// FileBoardPersistence
// - 파일에 게시판 목록을 저장하는 클래스
public class FileBoardPersistence {
  public void save() {
  }

  public void delete() {
  }
}
```

```java
// DbBoardPersistence
// - 데이터베이스에 게시판 목록을 저장하는 클래스
public class DbBoardPersistence {
  public void save() {
  }

  public void delete() {
  }
}
```

의존성 주입(DI, Dependency Injection)

- 객체를 생성할 때 의존하는 객체를 외부에서 주입하자!
- **1. 생성자**를 이용한 의존성 주입

```
// BoardService
// - 게시판 목록을 관리하는 기능 제공
public class BoardService {
  private final IBoardPersistence persistence; // 인터페이스

  // 생성자를 이용해 객체 주입
  public BoardService(IBoardPersistence persistence) {
    this.persistence = persistence;
  }

  // ...
}
```

```
public static void main(String[] args) {
  IBoardPersistence persistence = new DbBoardPersistence();
  BoardService service = new BoardService(persistence);
}
```

FileBoardPersistence 로 다시 바뀌어야 한다면,
main 메소드 측 코드만 변경하면 됨!

즉, BoardService 코드를 변경할 필요가 없어짐!

의존성 주입(DI, Dependency Injection)

- 객체를 생성할 때 의존하는 객체를 외부에서 주입하자!
- **2. setter**를 이용한 의존성 주입

```
// BoardService
// - 게시판 목록을 관리하는 기능 제공
public class BoardService {
  private IBoardPersistence persistence; // 인터페이스

  // setter 를 이용한 의존성 주입
     public void setIBoardPersistence(IBoardPersistence persistence) {
   this.persistence = persistence;
  }

  // ...
}
```

```
public static void main(String[] args) {
  IBoardPersistence persistence = new DbBoardPersistence();

  BoardService service = new BoardService();
  service.setIBoardPersistence(persistence);
}
```

마찬가지로 FileBoardPersistence 로 다시 바뀌어야 한다면, main 메소드 측 코드만 변경하면 됨!

즉, BoardService 코드를 변경할 필요가 없어짐!

복습. DI (Dependency Injection, 의존성 주입) 방법

방법 1. Field Injection (필드 주입)

```
@Autowired
private FieldService fieldService;
```

방법 2. Setter Injection (수정자 주입)

```
//Setter injection
private SetterService setterService;

@Autowired
public void setSetterService(SetterService setterService) {
    this.setterService = setterService;
}
```

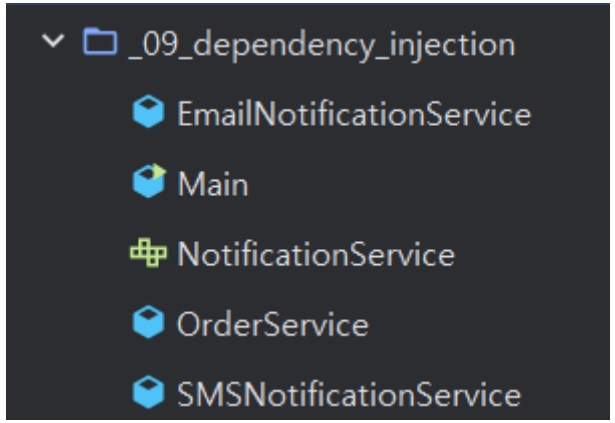
방법 3. Constructor Injection (생성자 주입)

```
//Constructor injection
private final ConstructorService constructorService;

@Autowired
public ExampleComponent(ConstructorService constructorService) {
    this.constructorService = constructorService;
}
```

실습. 온라인 쇼핑몰 주문 시스템 구현하기

- OrderService 클래스를 만들고, 이 클래스는 주문을 처리하는 processOrder() 메서드를 갖는다
- 주문 처리 후 알림을 보내는 기능을 구현하기 위해 NotificationService 인터페이스를 만든다
- NotificationService를 구현하는 EmailNotificationService와 SMSNotificationService 클래스를 만든다
- OrderService는 NotificationService에 의존한다
- 생성자 주입과 setter 주입 두 가지 방식으로 DI를 구현하기
  - EmailNotificationService: 생성자 주입 방식
  - SNSNotificatinService: setter 주입 방식
- Main 클래스에서 OrderService를 사용하여 주문을 처리하고 알림을 보내는 코드를 작성




```
Order processed successfully
Sending email notification: Your order has been processed
---
Order processed successfully
Sending SMS notification: Your order has been processed
```

실행 결과

스프링 프레임워크

- 프로젝트 규모가 커질수록 의존성 주입을 수동으로 관리하기 어려워짐
- 이 문제를 해결하고자 의존성 주입을 담당하는 컨테이너 필요해짐!

- 스프링 IoC 컨테이너
  - 스프링 프레임워크에서 의존성 주입을 담당하는 핵심 컴포넌트
  - 객체 생성시 주입해야 하는 객체를 자동으로 주입해줌 (단, Bean 으로 등록된 객체여야 함!)

- Bean
  - 스프링 컨테이너가 관리하는 객체로 애플리케이션의 핵심을 이룸
  - 스프링의 제어를 받음

- 스프링 컨테이너 (Bean Factory)
  - Bean의 생명주기를 관리하고 의존성을 주입하는 역할

정리. IoC 의미

예전에는 개발자가 개발자가 객체의 생성과 의존성 관리를 직접 했다면
스프링에서는 이 제어권이 컨테이너로 넘어감
➔ 객체 간의 결합도를 낮추고 코드 재사용성을 높임

Bean

Bean 이란?

- Spring IoC 컨테이너가 관리하는 Java 객체
- Spring 에서는 new 를 이용해 만들어진 객체가 아니라 Spring에서 관리하는 **Bean** 이라는 객체를 이용함

- 장점?
  - **의존성 관리**가 용이: Spring 이 객체의 생명주기와 의존관계를 관리하므로 개발자는 비즈니스 로직에 집중
  - **싱글톤(Singleton) 패턴**의 이점: 동일한 타입의 객체가 여러 번 생성되는 것이 아니라, 하나의 인스턴스를 생성하여 재사용 -> 메모리 사용을 절약, 애플리케이션 성능 향상
  - **AOP** 지원: Bean 으로 등록된 객체들을 Spring AOP를 통해 부가 기능을 손쉽게 추가 가능

Bean 등록 방법

- **@Component**
  - 가장 간단한 방법으로 클래스 선언부 위에 @Component 을 사용
  - 스프링에게 **"해당 클래스를 빈(Bean)으로 등록해줘"** 라고 알려주는 역할을 함
- 자바 설정 클래스를 직접 만들기
  - 설정 클래스를 만들고 @Configuration 이용
  - 해당 클래스 안의 메소드에 @Bean 사용해 개별 Bean 을 정의

- Bean 으로 등록된 객체를 의존성 주입받아 사용하려면? **@Autowired 사용**

Annotation

Annotation이란?

- Annotation(@) 는 사전적 의미로 주석
- Java에서 @ 로 시작하는 문법 요소로, 코드에 메타데이터를 추가하는 방법
  - 메타데이터: 데이터를 설명하는 데이터
- 코드 사이에 주석처럼 쓰이지만, 단순 주석과 달리 특별한 의미와 기능을 수행하게 함

- 예시
  - @Override: 메소드가 오버라이드 되었는지 검증
  - @Deprecated: 더 이상 사용되지 않음을 표시

Annotation 장점과 사용법

- 장점
  - 코드량 감소
  - 유지보수 쉬움
  - 생산성 증가

- 사용 순서
  - @ 를 정의한다.
  - 클래스에서 @를 배치한다.
  - 코드가 실행되는 중 추가 정보를 획득해 기능을 실시한다.

Spring Boot Annotation 종류

@SpringBootApplication

- Spring Boot 어플리케이션 실행 초기 진입점에 위치하는 어노테이션
- 애플리케이션의 구성을 자동으로 설정
- 애플리케이션 패키지 내의 모든 컴포넌트를 스캔하고 빈으로 등록
- 해당 어노테이션이 달린 클래스가 있는 패키지를 베이스 패키지로 간주

```
@SpringBootApplication
public class MyApplication {
  public static void main(String[] args) {
    SpringApplication.run(MyApplication.class, args);
  }
}
```

@Component

- 일반적인 스프링 관리 컴포넌트를 나타냄 *스프링 컴포넌트: 스프링 프레임워크에 의해 관리되는 객체
- 클래스를 스프링의 컴포넌트로 표시
- 컴포넌트 스캔 대상이 되어 자동으로 빈으로 등록됨
- @Service, @Repository, @Controller 의 기본이 되는 어노테이션
- 개발자가 직접 작성한 클래스를 스프링 관리하에 두기 위해 사용
- 즉, 간단하게 Bean 을 등록하기 위해 사용되는 Annotation

```
@Component
public class NotificationManager {
   public void sendNotification(String message) {
      // 알림 전송 로직
    }
}
```

@Controller

- 클래스를 스프링의 컨트롤러로 표시
- 요청 매핑 어노테이션 (@GetMapping, @PostMapping 등) 과 함께 사용해 HTTP 요청을 특정 메서드에 매핑
- 뷰 이름을 반환하거나 ResonseBody 를 사용하여 직접 응답 생성 가능

```
@Controller
public class HelloController {
    @GetMapping("/hello")
    public String hello(Model model) {
        model.addAttribute("message", "Hello, World!");
        return "hello";
    }
}
```

@RestController

- RESTful 웹 서비스의 컨트롤러를 정의
- @Controller, @ResponseBody 를 결합한 어노테이션
- 모든 요청 처리 메서드의 반환 값을 HTTP 응답 본문으로 직접 전송
- JSON 또는 XML 형식의 데이터를 쉽게 반환

```
@RestController
@RequestMapping("/api")
public class UserController {
    @GetMapping("/users/{id}")
    public User getUser(@PathVariable Long id) {
        // 사용자 조회 로직
            return new User(id, "John Doe");
    }
}
```

@Service

- 비즈니스 로직을 처리하는 서비스 클래스를 정의
- 클래스를 스프링의 서비스 컴포넌트로 표시
- 컴포넌트 스캔 대상이 되어 자동으로 빈으로 등록
- 주로 비즈니스 로직을 포함하는 클래스에 사용됨

```
@Service
public class EmailService {
    public void sendEmail(String to, String subject, String body) {
        // 이메일 전송 로직
    }
}
```

@Repository

- 데이터 접근 계층의 DAO(Data Access Object)를 정의
- 데이터베이스 예외를 스프링 데이터 접근 예외로 변환
- JPA 리포지토리와 함께 사용하면 쿼리 메소드를 자동으로 구현해줌

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
  List<User> findByLastName(String lastName);
}
```

@Autowired

- 스프링 컨테이너에 등록한 Bean 에게 의존 관계 주입이 필요할 때, DI (의존성 주입)을 가능하게 하는 어노테이션
- 스프링 컨테이너가 해당 타입의 빈을 찾아 자동으로 주입
- 생성자, 세터, 필드에 사용 가능

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public User findById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}
```

@RequestMapping

- HTTP 특정 요청을 특정 핸들러 메서드에 매핑하는데 사용
- 클래스 레벨과 메소드 레벨 모두에서 사용 가능
  - 클래스 레벨에서 사용 시 해당 컨트롤러의 모든 핸들러 메서드에 적용됨

```
@Controller
@RequestMapping("/users")
public class UserController {
    @RequestMapping(method = RequestMethod.GET)
    public String listUsers() {
        // 사용자 목록을 반환하는 로직
        return "userList";
    }

    @RequestMapping(value = "/{id}", method = RequestMethod.GET)
    public String getUser(@PathVariable Long id) {
        // 특정 사용자 정보를 반환하는 로직
        return "userDetails";
    }
}
```

@{HTTP_Method}Mapping

- @GetMapping: HTTP GET 요청 처리
  - @RequestMapping(method = RequestMethod.GET) → @GetMapping
- @PostMapping: HTTP POST 요청 처리
  - @RequestMapping(method = RequestMethod.POST) → @PostMapping
- @PutMapping: HTTP PUT 요청 처리
  - @RequestMapping(method = RequestMethod.PUT) → @PutMapping
- @PatchMapping: HTTP PATCH 요청 처리
  - @RequestMapping(method = RequestMethod.PATCH) → @PatchMapping
- @DeleteMapping: HTTP DELETE 요청 처리
  - @RequestMapping(method = RequestMethod.DELETE) → @DeleteMapping

@{HTTP_Method}Mapping

```java
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping
    public List<User> getAllUsers() {
        // 모든 사용자 조회
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        // 특정 사용자 조회
        return userService.findById(id);
    }

    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        // 새 사용자 생성
        User newUser = userService.create(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(newUser);
    }
```

```java
    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id, @RequestBody User user) {
        // 사용자 전체 정보 업데이트
        return userService.update(id, user);
    }

    @PatchMapping("/{id}")
    public User partialUpdateUser(@PathVariable Long id, @RequestBody Map<String, Object> updates) {
        // 사용자 정보 부분 업데이트
        return userService.partialUpdate(id, updates);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        // 사용자 삭제
        userService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

@RequestParam

- HTTP 요청 파라미터를 메서드 파라미터에 바인딩
- URL의 쿼리 파라미터나 폼 데이터를 메서드 파라미터로 받아옴
- 필수 여부, 기본 값 등을 지정 가능

```
@GetMapping("/search")
public String searchUsers(@RequestParam String name, @RequestParam(defaultValue = "1") int page) {
  // name과 page를 이용한 검색 로직
    return "searchResult";
}
```

@PathVariable

- URL 경로에 있는 변수 값을 추출하여 메서드 파라미터로 전달
- RESTful API 설계에서 자주 사용됨

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
  // id를 이용해 사용자 정보를 조회
    return userService.findById(id);
}
```

@ModelAttribute

- 요청 파라미터를 객체에 바인딩하고, 해당 객체를 모델에 추가
- HTTP 요청 파라미터를 지정된 객체의 프로퍼티에 바인딩
- 객체를 자동으로 모델에 추가하여 뷰에서 사용할 수 있게 함
- 폼 데이터를 받아 처리할 때 주로 사용됨

```
@PostMapping("/users")
public String createUser(@ModelAttribute User user) {
    userService.save(user);
    return "redirect:/users";
}
```

@Bean

- 스프링 컨테이너에 의해 관리될 빈 객체를 생성하는 메서드를 표시
- 메서드가 반환하는 객체를 스프링 컨테이너의 빈으로 등록
- 주로 @Configuration 클래스 내에서 사용됨
- 서드파티 라이브러리(개발자가 직접 제어가 불가능한 외부 라이브러리 등)의 클래스를 빈으로 등록할 때 유용

```
@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/mydb");
        dataSource.setUsername("user");
        dataSource.setPassword("password");
        return dataSource;
    }
}
```

@Congifuration

- 해당 클래스가 하나 이상의 @Bean 메서드를 선언하고 있음을 나타냄
- 클래스를 Spring 의 설정 클래스로 지정
- @Bean 어노테이션이 붙은 메서드를 통해 Bean 을 정의 가능

```
@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/mydb");
        dataSource.setUsername("user");
        dataSource.setPassword("password");
        return dataSource;
    }
}
```